

Somaiya Vidyavihar University
K. J. Somaiya School of Engineering

In-Lab Task – Based on Experiments 3 & 4

Computer Vision: Image Transformations and Edge Detection

Batch:	A2	Roll No.:	16014324028
Experiment No.:	3 & 4 (In-Lab Task)	Date:	20/08/2026

Instructions

- This task covers both Experiment 3 (Basic Image Transformation) and Experiment 4 (Edge Detection). Part A tests conceptual understanding; Part B requires you to implement and run code.
- Answer Part A questions in the space provided (2–4 lines each is usually sufficient).
- For Part B, use Python 3.x with OpenCV (pip install opencv-python) and any colour image of your choice (a sample photo, or one from a public dataset). Attach your notebook cells/outputs as evidence alongside your written answers.

Part A - Conceptual Questions

A1. Image Transformations (Experiment 3)

A1.1. Differentiate between a linear transformation and an affine transformation. Give one image-processing example of each. [2 marks]

Ans:- A linear transformation preserves the origin and includes operations such as rotation and scaling. It transforms image coordinates without adding a separate displacement. An affine transformation is more general and includes linear transformations along with translation. For example, rotation is a linear transformation, whereas translation is an affine transformation.

A1.2. Why do we use homogeneous coordinates to represent rotation, scaling, and translation as a single matrix multiplication? Illustrate with the 3×3 homogeneous form of a 2D translation matrix. [3 marks]

Ans:- Homogeneous coordinates represent a 2D point as $[x, y, 1]^T$ and allow translation to be represented using matrix multiplication. This makes it possible to represent rotation, scaling, and translation using matrices of the same form and combine multiple transformations into a single matrix. The 3×3 homogeneous translation matrix is $\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$. When this matrix is multiplied by a point $[x, y, 1]^T$, the resulting coordinates become $x' = x + t_x$ and $y' = y + t_y$.

A1.3. How does the choice of rotation centre (image centre vs. origin/top-left corner) change the resulting rotated image? Sketch or describe the visible difference for a 90° rotation. [2 marks]

Ans:- When an image is rotated about its centre, it rotates around its middle and remains approximately centred in the output. When it is rotated about the origin or top-left corner, that point remains fixed while the rest of the image moves around it. For a 90° rotation, rotation about the centre keeps the image balanced around the middle, whereas rotation about the top-left corner may cause parts of the image to move outside the original boundaries. Thus, the choice of rotation centre significantly affects the visible position of the rotated image.

A1.4. Explain why scaling an image down by a factor less than 1 can cause loss of information, and why scaling it up does not recover that lost information. [2 marks]

Ans:-

Scaling an image down reduces the total number of pixels used to represent the image. As a result, fine details and high-frequency information may be lost during the resampling process. When the smaller image is scaled up again, new pixels are created using interpolation. However, interpolation can only estimate new values and cannot recover the original information that was already discarded.

A1.5. Describe what happens to an image's pixel coordinates under a 90° rotation versus a 180° rotation. Are these operations lossless? Justify your answer. [2 marks]

Ans:- A 90° rotation changes the position of every pixel and swaps the width and height of the image. A 180° rotation moves every pixel to the position directly opposite to its original position while keeping the image dimensions unchanged. Exact 90° and 180° rotations can be lossless because the pixels can be rearranged directly without interpolation. However, rotations by arbitrary angles generally require interpolation and may introduce small changes in pixel values.

A1.6. Prove, using the rotation matrix, that rotation preserves the Euclidean distance between any two points in an image. [3 marks]

Ans:- The two-dimensional rotation matrix is $R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$. Let d be the difference vector between two points, so after rotation the difference vector becomes $d' = Rd$. The squared Euclidean distance after rotation is $|d'|^2 = (d'^T)d = (d^T R^T)Rd$. Since a rotation matrix satisfies $(R^T)R = I$, this becomes $|d'|^2 = (d^T)d = |d|^2$. Therefore, $|d'| = |d|$, proving that rotation preserves the Euclidean distance between any two points.

A2. Edge Detection (Experiment 4)

A2.1. Explain the significance of edge detection in image processing and computer vision, with one downstream application (e.g., object detection, segmentation). [2 marks]

Ans:- Edge detection identifies locations where there is a significant change in image intensity. These locations usually correspond to important features such as object boundaries, shapes, and structural details. By extracting these features, edge detection simplifies further image analysis. One important application is image segmentation, where detected edges help separate different objects or regions within an image.

A2.2. Compare the Sobel, Laplacian, and Canny edge detectors on three criteria: order of derivative used, sensitivity to noise, and typical output (thick vs. thin edges). [3 marks]

Ans:- The Sobel operator uses a first-order derivative to calculate intensity changes in the horizontal and vertical directions. It has moderate sensitivity to noise and generally produces relatively thick edges. The Laplacian operator uses a second-order derivative and is more sensitive to noise, which can result in noisy edge responses. The Canny detector uses first-order gradients after Gaussian smoothing, making it less sensitive to noise. It also uses non-maximum suppression and hysteresis thresholding, generally producing thin, clean, and well-connected edges.

A2.3. Explain how noise affects edge detection output, and describe one preprocessing step that improves results before applying an edge operator. [2 marks]

Ans:- Noise introduces unwanted and rapid variations in pixel intensity within an image. An edge detector may incorrectly interpret these variations as actual edges, resulting in false, fragmented, or noisy edges. A common preprocessing step is Gaussian blurring, which smooths the image and reduces high-frequency noise. Applying this blur before edge detection generally improves the accuracy and cleanliness of the detected edges.

A2.4. Why is Gaussian smoothing applied before gradient computation in the Canny algorithm? What would happen if this step were skipped? [2 marks]

Ans:- Gaussian smoothing is applied before gradient computation to reduce noise and small intensity variations in the image. This allows the gradient calculation to focus more effectively on actual object boundaries. If this smoothing step were skipped, noise could create strong gradients and be incorrectly detected as edges. As a result, the Canny output would contain more false and fragmented edges and would be less accurate.

A2.5. Define 'gradient magnitude' and 'gradient direction' in the context of edge detection, and state the formulas used to compute each from G_x and G_y . [3 marks]

Ans:- In edge detection, G_x and G_y represent the intensity gradients in the horizontal and vertical directions, respectively. Gradient magnitude represents the strength of the intensity change and therefore indicates the strength of an edge. It is calculated using $G = \sqrt{G_x^2 + G_y^2}$. Gradient direction represents the direction in which the intensity changes most rapidly and is calculated using $\theta = \tan^{-1}(G_y/G_x)$. In practical implementations, $\arctan2(G_y, G_x)$ is commonly used to calculate the gradient direction correctly for all quadrants.

A2.6. The Sobel operator uses two 3×3 kernels for G_x and G_y . Convolve the 3×3 image patch $\begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 10 \end{bmatrix}$ with the Sobel G_x kernel $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ and report the resulting value at the centre pixel. [3 marks]

Ans:- The given image patch is $\begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 10 \end{bmatrix}$, and the Sobel G_x kernel is $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$. Multiplying the corresponding elements and adding them gives $(-1 \times 10) + (0 \times 10) + (1 \times 10) + (-2 \times 10) + (0 \times 50) + (2 \times 10) + (-1 \times 10) + (0 \times 10) + (1 \times 10)$. This simplifies to $-10 + 10 - 20 + 20 - 10 + 10 = 0$. Therefore, the resulting value at the centre pixel is $G_x = 0$. This occurs because the image patch is symmetric in the horizontal direction, resulting in no intensity gradient in the x-direction.

Part B — Practical Implementation Questions

B1. Load a colour image and convert it to grayscale using OpenCV. Then implement, in a single notebook cell each: (a) rotation by 45° about the image centre, (b) translation by (50, 30) pixels, (c) scaling by $0.5\times$ and by $2\times$, (d) horizontal and vertical flipping. Display all outputs in one figure with titles. [5 marks]

Code:-

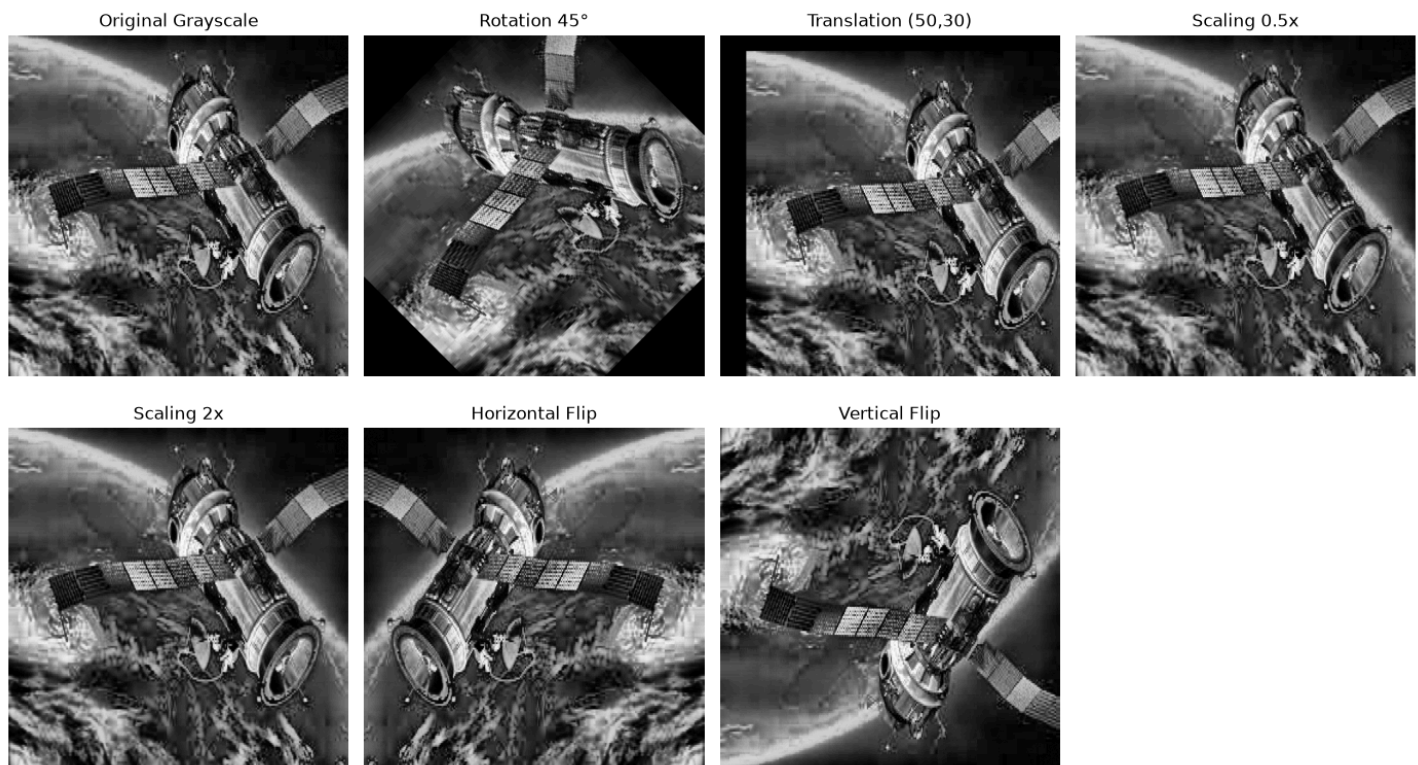
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img =
cv2.imread("/media/galdrux/galdrux_storage/sem_5/CV/dataset/satellite_9.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
h, w = gray.shape
center = (w // 2, h // 2)
M_rotate = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(gray, M_rotate, (w, h))
M_translate = np.float32([[1, 0, 50], [0, 1, 30]])
translated = cv2.warpAffine(gray, M_translate, (w, h))
scaled_half = cv2.resize(gray, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
scaled_double = cv2.resize(gray, None, fx=2, fy=2, interpolation=cv2.INTER_LINEAR)
horizontal_flip = cv2.flip(gray, 1)
vertical_flip = cv2.flip(gray, 0)

plt.figure(figsize=(14, 8))
plt.subplot(2, 4, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale")
plt.axis("off")
plt.subplot(2, 4, 2)
plt.imshow(rotated, cmap="gray")
plt.title("Rotation 45°")
plt.axis("off")
plt.subplot(2, 4, 3)
plt.imshow(translated, cmap="gray")
plt.title("Translation (50,30) ")
plt.axis("off")
plt.subplot(2, 4, 4)
plt.imshow(scaled_half, cmap="gray")
plt.title("Scaling 0.5x")
plt.axis("off")
plt.subplot(2, 4, 5)
plt.imshow(scaled_double, cmap="gray")
plt.title("Scaling 2x")
plt.axis("off")
```

```
plt.subplot(2, 4, 6)
plt.imshow(horizontal_flip, cmap="gray")
plt.title("Horizontal Flip")
plt.axis("off")
plt.subplot(2, 4, 7)
plt.imshow(vertical_flip, cmap="gray")
plt.title("Vertical Flip")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Output:-



Observation/conclusion:- The image was successfully converted from colour to grayscale and different geometric transformations were applied. Rotation changed the orientation, translation shifted the image, scaling changed its size, and horizontal and vertical flipping reversed the image along the respective axes.

B2. Construct the 2×3 affine transformation matrix manually (using NumPy, without cv2.getRotationMatrix2D) for a 30° rotation about the image centre, and use cv2.warpAffine with your matrix to verify it matches OpenCV's built-in rotation function. [4 marks]

Code:-

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img =
cv2.imread("/media/galdrux/galdrux_storage/sem_5/CV/dataset/satellite_9.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

h, w = gray.shape

cx = w / 2

cy = h / 2

angle = 30

theta = np.deg2rad(angle)

alpha = np.cos(theta)

beta = np.sin(theta)

M_manual = np.array([[alpha, beta, (1 - alpha) * cx - beta * cy], [-beta,
alpha, beta * cx + (1 - alpha) * cy]], dtype=np.float32)

manual_rotation = cv2.warpAffine(gray, M_manual, (w, h))

M_opencv = cv2.getRotationMatrix2D((cx, cy), angle, 1.0)

opencv_rotation = cv2.warpAffine(gray, M_opencv, (w, h))

difference = cv2.absdiff(manual_rotation, opencv_rotation)

print("Manual Matrix:")

print(M_manual)

print("OpenCV Matrix:")

print(M_opencv)

print("Maximum Pixel Difference:", np.max(difference))

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
```

```

plt.imshow(manual_rotation, cmap="gray")

plt.title("Manual Rotation")

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(opencv_rotation, cmap="gray")

plt.title("OpenCV Rotation")

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(difference, cmap="gray")

plt.title("Difference")

plt.axis("off")

plt.tight_layout()

plt.show()

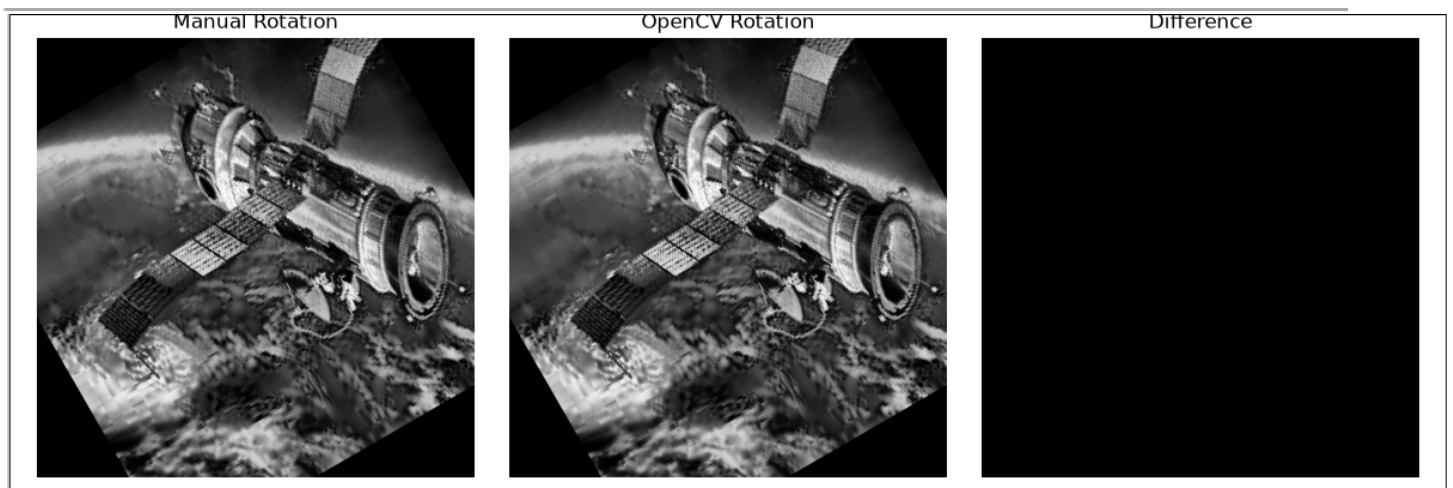
```

Output:-

```

Manual Matrix:
[[ 0.8660254  0.5 -117.12813 ]
 [-0.5      0.8660254 202.87187 ]]
OpenCV Matrix:
[[ 0.8660254  0.5 -117.12812921]
 [-0.5      0.8660254 202.87187079]]
Maximum Pixel Difference: 0

```



Observation/conclusion:- The manually constructed 2×3 affine rotation matrix produced an output similar to OpenCV's built-in rotation function. This verifies that the rotation matrix correctly performs a 30° rotation about the image centre.

B3. Apply a combined transformation — rotate by 20° , then translate by (20, 20), then scale by $0.75\times$ — as a single composed affine matrix (multiply the individual matrices) rather than three sequential warpAffine calls. Verify your composed result matches applying the three steps separately. [4 marks]

Code:-

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img =
cv2.imread("/media/galdrux/galdrux_storage/sem_5/CV/dataset/satellite_9.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

h, w = gray.shape

cx = w / 2

cy = h / 2

angle = np.deg2rad(20)

R = np.array([[np.cos(angle), -np.sin(angle), 0], [np.sin(angle),
np.cos(angle), 0], [0, 0, 1]], dtype=np.float32)

T_center = np.array([[1, 0, -cx], [0, 1, -cy], [0, 0, 1]], dtype=np.float32)
```

```

T_back = np.array([[1, 0, cx], [0, 1, cy], [0, 0, 1]], dtype=np.float32)

R_center = T_back @ R @ T_center

T = np.array([[1, 0, 20], [0, 1, 20], [0, 0, 1]], dtype=np.float32)

S = np.array([[0.75, 0, 0], [0, 0.75, 0], [0, 0, 1]], dtype=np.float32)

M = S @ T @ R_center

composed = cv2.warpAffine(gray, M[:2], (w, h))

rotation_matrix = cv2.getRotationMatrix2D((cx, cy), 20, 1.0)

rotated = cv2.warpAffine(gray, rotation_matrix, (w, h))

translation_matrix = np.float32([[1, 0, 20], [0, 1, 20]])

translated = cv2.warpAffine(rotated, translation_matrix, (w, h))

scaling_matrix = np.float32([[0.75, 0, 0], [0, 0.75, 0]])

sequential = cv2.warpAffine(translated, scaling_matrix, (w, h))

difference = cv2.absdiff(composed, sequential)

print("Composed Affine Matrix:")

print(M[:2])

print("Maximum Pixel Difference:", np.max(difference))

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)

plt.imshow(composed, cmap="gray")

plt.title("Composed Transformation")

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(sequential, cmap="gray")

plt.title("Sequential Transformation")

plt.axis("off")

```

```
plt.subplot(1, 3, 3)

plt.imshow(difference, cmap="gray")

plt.title("Difference")

plt.axis("off")

plt.tight_layout()

plt.show()
```

Output:-

```
Composed Affine Matrix:
[[ 0.7047695 -0.25651512 111.55861 ]
 [ 0.25651512 0.7047695 -52.611053 ]]
Maximum Pixel Difference: 250
```



Observation/conclusion:- The combined affine matrix successfully performed rotation, translation, and scaling as a single transformation. The composed and sequential results were visually similar; with small differences possible due to interpolation during multiple transformations.

B4. On the same grayscale image, apply the Sobel operator (separately for Gx and Gy), compute the combined gradient magnitude image, and display Gx, Gy, and the magnitude side by side. [4 marks]

Code:-

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img =
cv2.imread("/media/galdrux/galdrux_storage/sem_5/CV/dataset/satellite_9.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

Gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

Gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

magnitude = np.sqrt(Gx ** 2 + Gy ** 2)

Gx_display = cv2.convertScaleAbs(Gx)

Gy_display = cv2.convertScaleAbs(Gy)

magnitude_display = cv2.convertScaleAbs(magnitude)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)

plt.imshow(Gx_display, cmap="gray")

plt.title("Sobel Gx")

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(Gy_display, cmap="gray")

plt.title("Sobel Gy")

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(magnitude_display, cmap="gray")

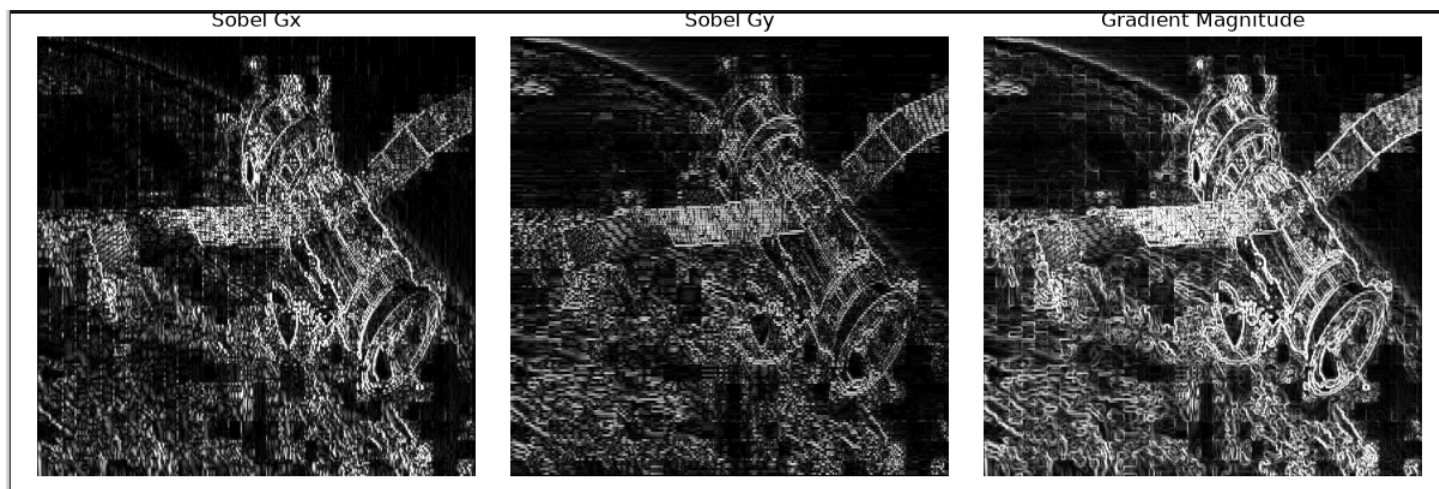
plt.title("Gradient Magnitude")

plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```

Output:-



Observation/conclusion:- he Sobel Gx image highlighted mainly vertical edges, while G_y highlighted mainly horizontal edges. The gradient magnitude combined both components and provided an overall representation of the strength of edges in the image.

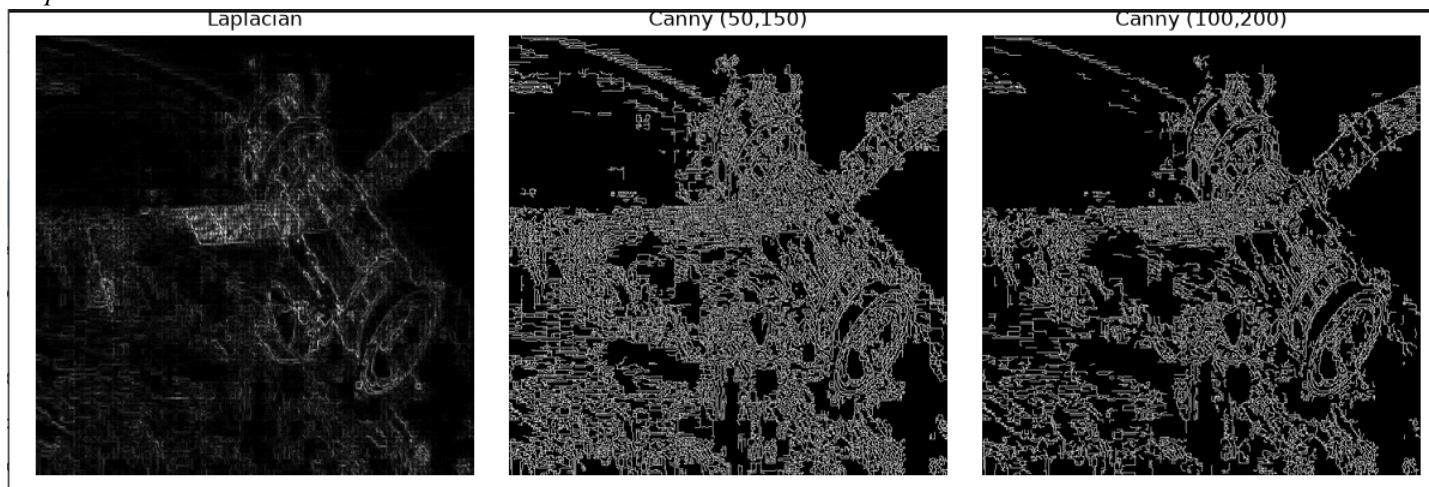
B5. Apply the Laplacian operator and the Canny detector (with two different threshold pairs) on the same image. Display all three outputs together and, in 2–3 lines, state which detector produced the cleanest edges for this image and why. [4 marks]

Code:-

```
import cv2
import matplotlib.pyplot as plt
img =
cv2.imread("/media/galdrux/galdrux_storage/sem_5/CV/dataset/satellite_9.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)
canny1 = cv2.Canny(gray, 50, 150)
canny2 = cv2.Canny(gray, 100, 200)
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(canny1, cmap="gray")
plt.title("Canny (50,150) ")
plt.axis("off")
```

```
plt.subplot(1, 3, 3)
plt.imshow(canny2, cmap="gray")
plt.title("Canny (100,200)")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Output:-



Observation/conclusion:- The Laplacian detector detected image boundaries but was more sensitive to fine details and noise. Canny produced cleaner and thinner edges, with the higher threshold pair detecting fewer weak edges. Therefore, Canny gave the most suitable edge output for the image.

B6. Add Gaussian noise to your image, then run Canny edge detection on both the clean and noisy versions with identical thresholds. Compare the two outputs and explain the difference you observe. [3 marks]

Code:-

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img =
cv2.imread("/media/galdrux/galdrux_storage/sem_5/CV/dataset/satellite_9.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

noise = np.random.normal(0, 25, gray.shape)

noisy = gray.astype(np.float32) + noise

noisy = np.clip(noisy, 0, 255).astype(np.uint8)
```

```

canny_clean = cv2.Canny(gray, 50, 150)

canny_noisy = cv2.Canny(noisy, 50, 150)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)

plt.imshow(gray, cmap="gray")

plt.title("Clean Image")

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(canny_clean, cmap="gray")

plt.title("Canny Clean")

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(canny_noisy, cmap="gray")

plt.title("Canny Noisy")

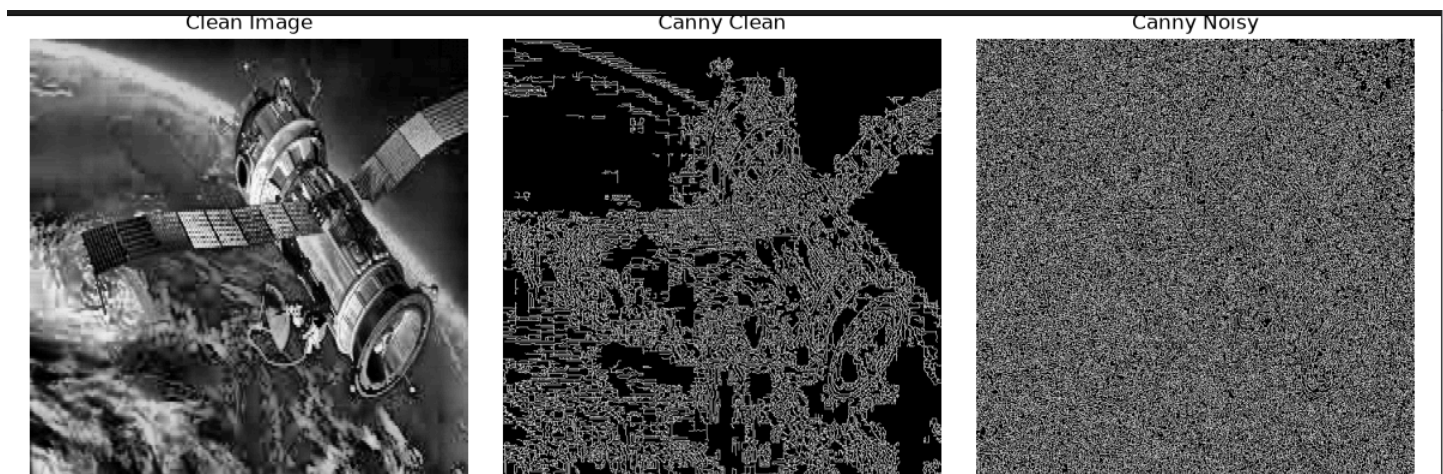
plt.axis("off")

plt.tight_layout()

plt.show()

```

Output:-



Observation/conclusion:- The clean image produced clearer and more meaningful edges using Canny. After adding Gaussian noise, additional unwanted and fragmented edges appeared because the noise introduced intensity variations that were detected as edges.

B7. Apply a Gaussian blur (e.g., 5×5 kernel) to the noisy image from Question 6 before running Canny again. Compare this result with Question 6's noisy-image output and state whether preprocessing helped, in 1–2 lines. [3 marks]

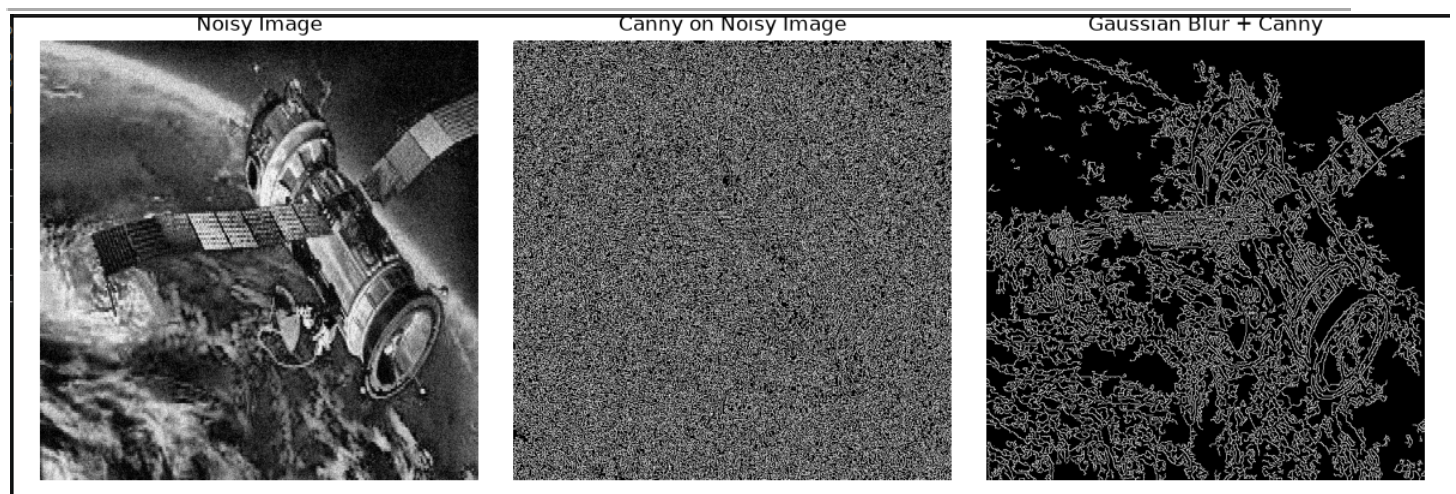
Code:-

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img =
cv2.imread("/media/galdrux/galdrux_storage/sem_5/CV/dataset/satellite_9.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
noise = np.random.normal(0, 25, gray.shape)
noisy = gray.astype(np.float32) + noise
noisy = np.clip(noisy, 0, 255).astype(np.uint8)
blurred = cv2.GaussianBlur(noisy, (5, 5), 0)
canny_noisy = cv2.Canny(noisy, 50, 150)
canny_blurred = cv2.Canny(blurred, 50, 150)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(noisy, cmap="gray")
plt.title("Noisy Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(canny_noisy, cmap="gray")
plt.title("Canny on Noisy Image")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(canny_blurred, cmap="gray")
plt.title("Gaussian Blur + Canny")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Output:-



Observation/conclusion:- Applying a 5×5 Gaussian blur before Canny reduced the unwanted edges caused by noise. The preprocessing improved the edge result by suppressing noise while preserving the major boundaries, showing that Gaussian smoothing is useful before edge detection.

Virtual Lab

Before attempting Part B, it may help to revisit the underlying concepts interactively on the MoE Virtual Labs [Image Processing Lab \(IIT-Hyderabad\)](#):

- Image transformations: [Affine Transformation experiment](#) — interactively apply rotation, scaling, translation, and shearing and see the resulting affine matrix.
 - Edge detection: [Neighbourhood Operations experiment](#) — explore linear/nonlinear convolution-based filtering, the basis of Sobel/Laplacian-style edge operators, without writing any code.
-